OOPS

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects that contain data (fields) and behavior (methods).

Example

- Object - car
- Attributes – color, brand, speed,
- Behaviours- Start(), Stop(), Accelerate()

Necessity of OOPS

- Improves code reusability
- Enhances maintainability and scalability
- Makes programs easier to understand and manage
- Closely models real-world entities

CLASS

A logical blueprint or template used to define the state (variables) and behavior (methods) of a specific entity.

OBJECT

A physical instance of a class that takes up memory space and contains real values.

Example

```
// The blueprint (Class)
class Car {
   String color; // State
   void drive() { System.out.println("Moving..."); } // Behavior
}

// The Reality (Object creation)
Car myCar = new Car();
```

INHERITANCE

Inheritance is a core OOP concept in Java that allows one class to acquire the fields and methods of another class using the extends keyword. It represents an “is-a” relationship between classes.

- The class being inherited is called the superclass, and the inheriting class is the subclass.
- A subclass can use existing features of the superclass and also add its own.
- Inheritance promotes code reusability and reduces redundancy.

Example:

Dog, Cat, Cow can be Derived Class of Animal Base Class.

ABSTRACTION

Abstraction in Java is the process of hiding implementation details and showing only the essential features of an object. It helps users focus on what an object does rather than how it does it.

- **Hides complexity:** Internal implementation details are hidden from the user.
- **Improves maintainability:** Changes in implementation do not affect the user code.
- **Enhances flexibility:** Supports loose coupling through abstract classes and interfaces.

Example:

An ATM or a coffee machine represents abstraction, where the user interacts with simple operations while the internal working and implementation details remain hidden

ENCAPSULATION

Encapsulation is the process of wrapping data and methods into a single unit, usually a class, and restricting direct access to the data. It acts as a protective shield that prevents data from being accessed directly from outside the class.

- Data members are hidden using the private access modifier.
- Access to data is provided through public getter and setter methods.
- It improves data security, maintainability, and controlled access.

Polymorphism

Polymorphism means “many forms”, where a single entity can behave differently in different situations. In Java, it allows the same method or object to show different behavior based on context.

- Same method, different behavior depending on the object
- Achieved through method overloading and method overriding

Example:

Different animals represent polymorphism, where the same method speak() produces different outputs like Bark, Meow, and Moo depending on the object.